

SIMATS ENGINEERING
ASSESSMENT TOOL 1: Constraint-Based Problem Solving

COURSE NAME: COMPUTER VISION

COURSE CODE: ITA0510

Slot A

COURSE OUTCOME COVERED

CO5: Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos. (BTL 6)

Assessment Tool Weightage: 40%

QUESTIONS:5

Total Marks 50

Duration :60 Minutes

Annexure A
Constraint-Based Problem Solving

Code of Conduct:

I _____M.THANUJA(192425356)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. IL= understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: M.THANUJA

Rubrics for Evaluation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks(100)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly	
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization	
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints	
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts	

Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided	
---------------	-----	--	--	-------------------------------	---------------------------	--

Constraint-Based Problem Solving: Analyze the problem and design an end-to-end computer vision pipeline using OpenCV, clearly identifying each stage from input acquisition to final output. Ensure that your solution satisfies all given constraints and justify your design decisions at each stage.

1. Facial Expression Recognition for Mobile Application

A mobile app must detect facial expressions (happy, sad, neutral) with the following constraints:

- A. Limited processing power (mobile CPU)**
- B. Real-time performance**
- C. Robustness to slight head movement and lighting changes**

Design a solution using OpenCV for face detection and expression analysis. Justify how your approach balances accuracy and efficiency.

QUESTION 1: FACIAL EXPRESSION RECOGNITION FOR MOBILE APPLICATION

Problem Statement

The objective is to design a mobile application that detects and classifies facial expressions into **Happy, Sad, and Neutral** using OpenCV. The system must operate with **limited mobile CPU**, provide **real-time performance**, and remain robust against **slight head movements and lighting changes**.

1. Problem Understanding

The application receives live images from the mobile camera and performs the following tasks:

1. Capture the image from the camera.
2. Detect the face.
3. Extract the facial region.
4. Analyze facial features.
5. Classify the expression.
6. Display the expression on the screen.

2. Constraints Analysis

Constraint	Approach
Limited mobile CPU	Use lightweight OpenCV operations
Real-time performance	Reduce image size and avoid unnecessary processing
Head movement	Use face tracking between detections
Lighting changes	Grayscale conversion and contrast normalization
Accuracy	Use important facial features for classification

3. Proposed OpenCV Pipeline

Camera Input

↓

Frame Resizing

↓

Grayscale Conversion

↓

Face Detection

↓

Face ROI Extraction

↓

Facial Feature Extraction

↓

Expression Classification

↓

Display Result

Stage 1 – Image Acquisition

Capture live frames using the mobile camera.

Stage 2 – Preprocessing

Resize the frame to reduce computational requirements and convert it into grayscale.

```
frame = cv2.resize(frame, (640, 480))
```

```
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
```

Stage 3 – Face Detection

OpenCV's face detection method can be used to locate the face.

```
faces = face_cascade.detectMultiScale(  
    gray, scaleFactor=1.1, minNeighbors=5  
)
```

Stage 4 – Facial Feature Extraction

The detected face is extracted as a Region of Interest (ROI). Features such as:

- Eye openness
- Mouth shape
- Eyebrow position
- Mouth curvature

can be analyzed.

Stage 5 – Expression Classification

The extracted features are classified into:

- **Happy**
- **Sad**
- **Neutral**

Stage 6 – Display

The face is marked with a bounding box and the detected expression is displayed.

4. Handling Constraints

Limited CPU

Use:

- Low-resolution images
- Grayscale processing
- Lightweight detection
- Frame skipping

Real-Time Performance

Process only selected frames and track the face between detections.

Head Movement

Face tracking helps maintain the detected face even when the user moves slightly.

Lighting Changes

Grayscale conversion and contrast enhancement help reduce the effect of illumination changes.

5. Justification

The proposed solution balances accuracy and efficiency by reducing the amount of image data processed and using lightweight OpenCV operations. Face detection can be performed periodically, while tracking handles intermediate frames.

Final Answer

Therefore, the proposed pipeline provides an efficient solution for **real-time facial expression recognition on a mobile device** while satisfying the constraints of CPU usage, head movement, and lighting variation.

2. Smart Traffic Monitoring

System A city authority needs a system to:

- A. Detect vehicles from live video streams**
- B. Track movement across frames**
- C. Count vehicles and display statistics**
- D. Operate continuously with minimal delay**

Design a complete OpenCV-based system including video handling, detection, tracking, and visualization (drawing functions). Justify how your system ensures accuracy and real-time performance.

QUESTION 2: SMART TRAFFIC MONITORING SYSTEM

Problem Statement

The system must detect vehicles from live video, track their movement across frames, count vehicles, display statistics, and operate continuously with minimum delay.

1. Problem Understanding

The system performs four major operations:

1. Vehicle detection
2. Vehicle tracking
3. Vehicle counting
4. Statistics visualization

2. Constraints Analysis

Constraint	Approach
Live video	Continuous frame acquisition
Vehicle detection	Object detection
Movement tracking	Object tracking
Vehicle counting	Virtual counting line
Minimal delay	Efficient frame processing
Visualization	OpenCV drawing functions

3. Proposed Pipeline

Live Video

↓

Frame Acquisition

↓

Preprocessing

↓

Vehicle Detection

↓

Object Tracking

↓

Vehicle Counting

↓

Statistics Generation

↓

Visualization

4. Solution Development

Stage 1 – Video Acquisition

Read frames continuously from the traffic camera.

```
cap = cv2.VideoCapture("traffic.mp4")
```

Stage 2 – Preprocessing

Resize the video frames to reduce processing time.

Stage 3 – Vehicle Detection

Detect vehicles such as:

- Cars
- Buses
- Trucks
- Motorcycles

Each detected vehicle receives a bounding box.

Stage 4 – Vehicle Tracking

Assign a unique ID to every detected vehicle.

Example:

Vehicle ID 1

Vehicle ID 2

Vehicle ID 3

The tracking system maintains the identity of each vehicle across consecutive frames.

Stage 5 – Vehicle Counting

A virtual line is created on the road.

=====

COUNTING LINE

=====

When a vehicle crosses the line, the counter is increased.

Vehicle Count = Vehicle Count + 1

Each vehicle must be counted only once.

Stage 6 – Visualization

Use OpenCV drawing functions to display:

- Bounding boxes
- Vehicle IDs
- Counting line
- Total vehicle count
- Processing information

5. Accuracy and Real-Time Performance

Accuracy

Accuracy can be improved by:

- Using a suitable confidence threshold
- Maintaining vehicle IDs
- Using a virtual counting line
- Preventing duplicate counting

Real-Time Performance

Performance can be improved by:

- Resizing frames
- Processing only the required road region
- Skipping unnecessary frames
- Using efficient tracking

6. Justification

Object detection identifies vehicles, while tracking maintains their identity across frames. The virtual line provides an efficient method for counting vehicles without repeatedly counting the same vehicle.

Final Answer

Thus, the proposed OpenCV-based traffic monitoring system can continuously **detect, track, count, and visualize vehicles** while maintaining accuracy and minimizing processing delay.

3. Real-Time Object Detection on Mobile

Device A mobile application must:

- Detect objects using the camera**
- Run in real time**
- Use limited CPU and memory**

Design an OpenCV-based pipeline from image acquisition to detection and display. Justify how your design ensures efficiency under constraints.

QUESTION 3: REAL-TIME OBJECT DETECTION ON MOBILE DEVICE

Problem Statement

The mobile application must detect objects through the camera, operate in real time, and use limited CPU and memory.

1. Problem Understanding

The system should:

1. Capture camera frames.
2. Detect objects.
3. Identify object classes.
4. Locate objects.
5. Display the results in real time.

2. Constraints Analysis

Constraint	Solution
Limited CPU	Lightweight detection
Limited memory	Small input resolution
Real-time operation	Frame skipping
Camera processing	Continuous frame acquisition
Accuracy	Confidence threshold

3. Proposed Pipeline

Mobile Camera

↓

Frame Capture

↓

Resize

↓

Preprocessing

↓

Object Detection

↓

Confidence Filtering

↓

Bounding Box & Label

↓

Display

4. Solution Development

Stage 1 – Image Acquisition

`cap = cv2.VideoCapture(0)`

Frames are continuously captured.

Stage 2 – Resizing

Resize the image to a smaller resolution.

`frame = cv2.resize(frame, (320, 320))`

This reduces CPU and memory requirements.

Stage 3 – Object Detection

The detection model identifies objects and provides:

- Object class
- Confidence score
- Bounding box

Example:

Person → 94%

Bottle → 89%

Chair → 86%

Stage 4 – Confidence Filtering

Only detections above a selected confidence threshold are accepted.

$\text{Confidence} > \text{Threshold}$

↓

Accept Detection

Otherwise, the detection is ignored.

Stage 5 – Visualization

Display the bounding box and object name.

`cv2.rectangle(...)`

`cv2.putText(...)`

5. Efficiency Techniques

1. Reduce Image Resolution

Smaller images require fewer calculations.

2. Lightweight Detection

Use a model suitable for mobile devices.

3. Frame Skipping

Instead of detecting every frame:

Frame 1 → Detect

Frame 2 → Skip

Frame 3 → Detect

Frame 4 → Skip

4. Region of Interest

Process only the important region when possible.

5. Memory Optimization

Avoid storing unnecessary frames and reuse buffers.

6. Justification

The proposed design reduces CPU and memory consumption by using smaller images, lightweight processing, and selective frame detection.

Final Answer

Therefore, the proposed OpenCV pipeline provides **efficient real-time object detection on a mobile device** while satisfying CPU, memory, and processing constraints.

4. Smart Attendance System using Face

Recognition A system must:

- A. Capture faces from a webcam
- B. Recognize registered users
- C. Mark attendance automatically

Design a complete pipeline using OpenCV. Justify each stage and how the system avoids false recognition.

QUESTION 4: SMART ATTENDANCE SYSTEM USING FACE RECOGNITION

Problem Statement

The system must capture faces from a webcam, recognize registered users, automatically mark attendance, and avoid false recognition.

1. Problem Understanding

The system consists of:

1. Face capture
2. Face detection
3. Feature extraction
4. Face recognition
5. Identity verification
6. Attendance recording

2. Constraints Analysis

Constraint	Solution
Webcam input	Continuous frame capture
Face recognition	Compare with registered faces
False recognition	Recognition threshold
Duplicate attendance	Check attendance database
Real-time operation	Efficient processing

3. Proposed Pipeline

Webcam

↓

Frame Capture

↓

Preprocessing

↓

Face Detection

↓

Face ROI

↓

Feature Extraction

↓

Face Recognition

↓

Threshold Verification

↓

Attendance Recording



Display

4. Solution Development

Stage 1 – Face Registration

Capture face images of registered students.

Student



Capture Face



Detect Face



Extract Features



Store in Database

Stage 2 – Webcam Capture

```
cap = cv2.VideoCapture(0)
```

Stage 3 – Face Detection

Detect faces in the camera frame.

```
faces = face_cascade.detectMultiScale(gray)
```

Stage 4 – Feature Extraction

Extract important facial characteristics from the detected face.

Stage 5 – Face Recognition

Compare the captured face features with the registered users.

Captured Face



Feature Extraction



Database Comparison



Matching User

Stage 6 – False Recognition Prevention

A threshold is used to determine whether the match is reliable.

Valid Match



Recognized

Invalid/Low Confidence



Unknown

Attendance should not be recorded for an uncertain match.

Stage 7 – Attendance Recording

When the person is correctly recognized:

Student Name

+

Date

+

Time



PRESENT

The system also checks whether attendance has already been recorded.

5. Accuracy Improvements

False recognition can be reduced by:

- Using multiple face images during registration.
- Maintaining good lighting conditions.
- Applying preprocessing.
- Using an appropriate recognition threshold.
- Rejecting low-confidence matches.
- Preventing duplicate attendance.

6. Justification

The recognition threshold prevents uncertain faces from being accepted. The attendance database prevents multiple attendance entries for the same student.

Final Answer

Thus, the proposed OpenCV-based attendance system provides an automated method for **face detection, recognition, verification, and attendance marking**, while reducing false recognition.

5. Motion Detection for Home

Security A home security system must:

- A. Detect motion from a camera feed**
- B. Trigger alerts only for significant movement**
- C. Reduce false alarms**

Design the pipeline and justify how your approach handles noise and background variations.

QUESTION 5: MOTION DETECTION FOR HOME SECURITY

Problem Statement

The home security system must detect motion from a camera feed, generate alerts only for significant movement, and reduce false alarms caused by noise and background variations.

1. Problem Understanding

The system continuously monitors a camera and determines whether meaningful movement is present.

The main tasks are:

1. Capture video.
2. Detect changes.
3. Remove noise.
4. Identify significant movement.
5. Trigger an alert.
6. Reduce false alarms.

2. Constraints Analysis

Constraint	Solution
-------------------	-----------------

Continuous camera feed	Real-time frame processing
------------------------	----------------------------

Constraint	Solution
Image noise	Gaussian blur
Small changes	Thresholding
Background variation	Background subtraction
False alarms	Minimum area threshold
Significant motion	Contour analysis
Real-time requirement	Lightweight OpenCV operations

3. Proposed Pipeline

Camera Feed



Frame Acquisition



Grayscale Conversion



Gaussian Blur



Background Subtraction



Thresholding



Morphological Operations



Contour Detection



Area Filtering



Motion Detection



Alert

4. Solution Development

Stage 1 – Camera Acquisition

```
cap = cv2.VideoCapture(0)
```

Stage 2 – Grayscale Conversion

```
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
```

Grayscale reduces computational complexity.

Stage 3 – Noise Reduction

Apply Gaussian blur:

```
blur = cv2.GaussianBlur(gray, (21,21), 0)
```

This reduces small image variations.

Stage 4 – Background Subtraction

Compare the current frame with the background.

Current Frame

↓

Compare with Background

↓

Foreground / Motion Mask

Stage 5 – Thresholding

Convert the difference into a binary image.

Small Difference → 0

Large Difference → 255

Stage 6 – Morphological Operations

Dilation and erosion can be used to remove small noise and improve the detected motion regions.

Stage 7 – Contour Detection

Identify regions containing movement.

```
contours, _ = cv2.findContours(...)
```

Stage 8 – Area Filtering

Small regions are ignored.

Contour Area > Minimum Area

↓

Significant Motion

This is one of the most important steps for reducing false alarms.

Stage 9 – Alert Generation

If significant motion is detected:

MOTION DETECTED

An alert can then be generated.

5. Reducing False Alarms

Noise

Use Gaussian blur.

Small Movements

Use minimum contour-area filtering.

Background Changes

Use background subtraction.

Lighting Changes

Use suitable preprocessing and thresholding.

Repeated Motion

Require movement to persist for a few frames before generating an alert.

6. Justification

The combination of **background subtraction, thresholding, morphological operations, and contour-area filtering** allows the system to distinguish significant movement from small image variations.

Final Answer

Therefore, the proposed OpenCV pipeline provides an effective **real-time home security motion detection system** with reduced false alarms and efficient processing.